

Universidade do Vale do Itajaí

Escola Politécnica

Engenharia de Computação

Sistemas em Tempo Real

**Análise temporal de um autopiloto didático
em ESP32 com FreeRTOS**

Avaliação M1 — Modelagem de Sistema

Temática: Drone

Autor:

Miguel Bertozzin

Professor:

Prof. Dr. Felipe Viel

Itajaí – SC

2026

Resumo

Este trabalho implementa e avalia um autopiloto didático em ESP32 com FreeRTOS, composto por fusão sensorial periódica, controle de atitude, navegação e segurança. Sensores touch geram comandos de rota, telemetria, estresse e emergência, enquanto sensores inerciais e motores são simulados. Foram analisados 18 ensaios em um núcleo, com duração de 30,000 a 35,015 s, abrangendo prioridades Deadline Monotonic (DM), customizadas e uma adaptação Rate Monotonic (RM), configurações preemptiva e cooperativa, frequências de 80, 160 e 240 MHz e dois ensaios adicionais sem time slicing. O processamento de 488 linhas de métricas permitiu comparar perdas de deadline, resposta, latência de ativação, jitter e ocupação aproximada das tarefas. CUSTOM e DM preemptivos a 160 e 240 MHz não registraram perdas nesta bateria. A 80 MHz, todas as condições avaliadas apresentaram perdas de navegação, e DM/RM preemptivos registraram atraso na conclusão do job de segurança. Os ensaios cooperativos registraram perdas da fusão inclusive a 240 MHz. A interpretação considera as limitações da referência temporal da fusão, da medição de duração com interferência e da quantidade de eventos de emergência. Os resultados evidenciam o compromisso entre prioridade da segurança, proteção do controle e capacidade de processamento, sem constituir garantia formal de pior caso.

Palavras-chave: sistemas em tempo real; ESP32; FreeRTOS; escalonamento; deadline; instrumentação temporal.

Sumário

Resumo	i
1 Introdução	1
2 Enunciado técnico selecionado	1
3 Implementação e modelo temporal	4
3.1 Arquitetura e operações simuladas	4
3.2 Touch, interrupções e estresse	6
3.3 Períodos, deadlines e criticidade	7
3.4 Políticas e configuração do escalonador	7
4 Metodologia experimental e processamento	8
4.1 Matriz e protocolo	8
4.2 Referências temporais e métricas	9
4.3 Agregação, CPU e rastreabilidade	9
5 Resultados e discussão	10
5.1 Comparação geral	10
5.2 Prioridades: CUSTOM e DM	11
5.3 Resposta da emergência	12
5.4 Preempção, cooperação e jitter	13
5.5 Frequência e extra de 160 MHz	14
5.6 Extra de time slicing	16
5.7 Extra RM	17
5.8 Defasagem entre fusão e controle	19
6 Limitações e atendimento aos requisitos	20
6.1 Consistência da instrumentação	20
6.2 Cobertura do enunciado	21
7 Conclusão	22
Referências	23
A Métricas completas por tarefa	24
B Rastreabilidade e material suplementar	30

Lista de Figuras

1	Arquitetura do autopiloto didático.	5
2	Perdas de deadline por tarefa na matriz principal.	12
3	Respostas máximas e deadlines na matriz principal.	13
4	Decomposição da resposta de segurança nos 18 ensaios.	14
5	Jitter máximo reportado na matriz principal.	15
6	Evolução das perdas da FUS nas configurações cooperativas.	15
7	Efeito observado da frequência no custo base e na emergência.	16
8	Ocupação média aproximada atribuída às quatro tarefas.	17
9	Comparação exploratória de time slicing em DM preemptivo.	18
10	DM e RM: comparação das taxas reportadas.	18
11	Diferença de jobs entre FUS e CTRL no snapshot final.	19
12	Auditoria da referência temporal da FUS.	20

Lista de Tabelas

1	Entradas touch utilizadas.	6
2	Quantidade de trabalho artificial na versão consultada.	6
3	Modelo temporal adotado.	7
4	Prioridades implementadas; maior número indica maior prioridade.	7
5	Comparação dos 18 cenários: configuração, perdas, segurança e CPU.	10
6	Perdas por tarefa: misses/jobs e percentual acumulado.	11
7	Síntese do atendimento técnico ao enunciado.	21
8	Métricas finais de FUS_IMU: tempos em ms e CPU média em porcentagem.	24
9	Métricas finais de CTRL_ATT: tempos em ms e CPU média em porcentagem.	25
10	Métricas finais de NAV_PLAN: tempos em ms e CPU média em porcentagem.	26
11	Métricas finais de FAIL_SAFE: tempos em ms e CPU média em porcentagem.	27
12	Timestamps do último job e CPU da última janela por cenário/tarefa.	28
13	Correspondência entre IDs e arquivos de coleta.	30

1 Introdução

Em um sistema de tempo real, a utilidade de uma resposta depende de seu resultado e do instante em que ela é produzida. Em um autopiloto, a atualização do estado inercial alimenta o controle dos atuadores, enquanto comandos de navegação e eventos de segurança concorrem pelo processador. A prioridade atribuída a cada função e a disponibilidade de CPU influenciam diretamente esse encadeamento.

O presente trabalho desenvolve a temática Drone da Avaliação M1 de Sistemas em Tempo Real [1]. Implementou-se um autopiloto didático em C para ESP32, utilizando o ESP-IDF v6.1 e o FreeRTOS [2, 3]. O código disponibilizado pelo professor foi a referência inicial para a divisão em quatro tarefas [4]; a aplicação utilizada nos ensaios foi desenvolvida em `main.c`, com operações simuladas, sensores touch reais e instrumentação temporal. O exemplo oficial de touch do ESP-IDF e sua documentação fundamentaram a integração do periférico [5, 6].

Foram implementados geração de sinais inerciais e filtro complementar, controle PID com quatro comandos de motor simulados, atualização de waypoints, telemetria e modo seguro acionado por interrupção. A carga computacional foi complementada por laços com número fixo de operações. As prioridades foram selecionadas em compilação, permitindo comparar Deadline Monotonic (DM), uma política customizada com prioridade para a segurança e uma adaptação Rate Monotonic (RM).

A avaliação abrange 18 execuções em um núcleo, incluindo as frequências obrigatórias de 80 e 240 MHz, o extra de 160 MHz, os modos preemptivo e cooperativo e dois ensaios preemptivos sem time slicing. O objetivo é comparar perdas de prazo, duração dos trechos, latência, resposta, jitter e ocupação aproximada das tarefas. O processamento posterior em Python gerou tabelas e figuras a partir dos registros preservados, sem modificar as medições.

O documento apresenta os requisitos técnicos, as decisões de implementação, a metodologia de coleta e a discussão dos resultados. As métricas completas e os timestamps finais por tarefa são apresentados nos apêndices. As conclusões distinguem as observações experimentais de uma garantia formal de pior caso.

2 Enunciado técnico selecionado

Os trechos a seguir transcrevem o conteúdo técnico aplicável à temática Drone e à metodologia comum do enunciado [1]. Foram preservados os valores e a redação, com adaptação apenas de diagramação. Foram omitidos a temática Esteira, as regras administrativas de entrega e o exemplo ilustrativo de tabela, substituído neste relatório pelos resultados efetivamente coletados. A cópia integral do documento original acompanha o projeto em `fontes/enunciado.pdf`.

Linguagem e desenvolvimento

- Deverá ser usada as linguagens C/C++ para o desenvolvimento.
- Não será permitido o uso de bibliotecas ou funções prontas e disponibilizadas em base de dados ou em outros meios, salvo situações expressas no trabalho.

Temática 1 - Drone

O contexto dessa temática implementa, em uma ESP32 com FreeRTOS, um autopi-loto didático que reproduz o ciclo essencial de um drone: fusão sensorial periódica, controle de atitude, navegação e rotinas de segurança. Para tornar o experimento reproduzível em laboratório, todos os estímulos externos são simulados por sensores touch presente na própria ESP32, mapeados para “nova amostra inercial”, “co-mando de rota/telemetria” e fail-safe (emergência). Com isso, investigamos como restrições temporais rígidas (hard) e brandas (soft) se comportam sob diferentes políticas de prioridade (Rate Monotonic, Deadline Monotonic e um critério custo-mizado), além das configurações do escalonador.

O foco está na previsibilidade temporal: a tarefa periódica de fusão ($T=5$ ms) alimenta o controle de atitude com deadline curta, enquanto eventos de navega-ção/telemetria são assíncronos e o fail-safe exige baixa latência (gerada a partir de uma ISR que ativa task). O desempenho é medido por misses de deadline, jitter e la-tência ao toque, variando preempção/time slicing, frequência de CPU (80/160/240 MHz) e execução em uncore. O resultado permite discutir trade-offs entre robustez do controle, resposta a emergências e custo de preempções no tempo de execução.

Introdução

O sistema de controle do drone utiliza uma tarefa periódica para manter o estado inercial (fusão de sensores simulada) e tarefas encadeadas para controle de atitude. Eventos de navegação, telemetria e fail-safe são gerados por toques em pads touch. Todo o fluxo é instrumentado para medir latências e verificar deadlines sob diferentes políticas de prioridade e configurações do FreeRTOS.

Mapeamento de entradas (touch)

- Touch B: comando de rota (evento de navegação).
- Touch C: solicitação de telemetria/flush.
- Touch D: fail-safe/emergência.
- Touch A (opcional): injeta perturbação nos sinais para estresse de carga.

Tasks

1. FUS_IMU – Hard RT (analisar se é hard ou soft) – Periódica

- T (período): 5 ms; D: 5 ms; C (WCET): 1,0 ms (deve ser medido)

- Ação: filtragem/estimador (complementar/Madgwick) produz estado inercial.
 - Observação: produz amostra/estado e sinaliza a próxima task (queue/-notify).
2. **CTRL_ATT – Hard RT (analisar se é hard ou soft) – Encadeada (por FUS_IMU)**
- Gatilho: notificação/queue da FUS_IMU.
 - D: 5 ms após a amostra; C: ~0,8 ms (deve ser medido)
 - Ação: PID e atualização de atuadores (simulados).
3. **NAV_PLAN – Soft RT (analisar se é hard ou soft) – Evento (Touch B)**
- D: 20 ms; C: 3–4 ms (deve ser medido)
 - Ação: atualiza waypoint/rota; se flag de telemetria (Touch C), envia estado.
4. **FS_TASK – Hard RT – Evento (Touch D)**
- D: 10 ms desde o toque; C: 0,8–1,0 ms
 - Ação: pouso/hover seguro; reduz throttle imediatamente.

Metodologia de teste

1. Prioridades (três políticas):
 - (opcional – pontuação extra de 0,5 à 1,0) RM: menor período (ou maior taxa de eventos esperados) = maior prioridade.
 - DM: menor deadline = maior prioridade (útil pelas tarefas de segurança).
 - Custom: Investigar as tarefas mais críticas e atribuir prioridades.
2. Escalonador (FreeRTOSConfig.h):
 - Testar preemptivo vs cooperativo (`configUSE_PREEMPTION 1/0`).
 - (opcional) Em preemptivo, ligar/desligar time slicing (`configUSE_TIME_SLICING 1/0`).
 - Executar uma bateria curta (≥ 30 s por cenário) com bursts de toques (pressione A/B/C/D em sequências rápidas e/ou ao mesmo tempo).
3. Hardware (menuconfig):
 - Fixar UNICORE (Executar o FreeRTOS somente em um core causa diferença na execução).
 - Variar frequência de CPU (80/160/240 MHz).
 - Repetir cenários das políticas/escalonadores para cada frequência com o 80 MHz e 240 MHz.

- Extra (de 0,5 à 1,0 ponto): fazer a repetição com 160 MHz.
4. Critérios de sucesso:
 - Todos os deadlines foram atendidos durante vários testes
 5. Coleta e relatório:
 - Logar `timestamp_evento`, `timestamp_task_start`, `timestamp_task_end`.
 - Calcular misses, latência da IS ativar a task e ocupação de CPU.
 - Tabela comparativa por cenário (política $\times$ preempção $\times$ freq.)

Itens importantes

1. Você(s) deverá(ão) desenvolver o software requisitado pela empresa, seguindo a especificação sobre sensores, controladores e threads.
2. Deverá ser usado os sensores touch da ESP32, conforme indicado na imagem a seguir, para gerar as interrupções. Observe que os sensores então destacadas em cor roxa e iniciando com TOUCH. Há um total de 10 sensores.
3. Com o software, vocês deverão fazer uma análise temporal criteriosa do sistema desenvolvido. Poderão usar as bibliotecas indicadas nesse link: General Purpose Timer.
4. Devem ser identificadas a tasks com requisitos temporais hard e soft, e descrever o que levou a essa análise. Além disso, as operações podem ser consideradas sem requisito temporal (essa escolha deve ser embasada). Lembre-se de atualizar as prioridades de execução das threads do sistema no qual você faz uso.
5. Você pode simular as operações de código com geração de prints ou padrões de liga/desliga do LED na placa.

Nota de transcrição: a menção à imagem dos pinos remete à figura do PDF original; o mapeamento efetivamente utilizado é apresentado na Tabela 1. A expressão “latência da IS” foi mantida conforme o enunciado e é interpretada na implementação como a latência desde a ISR até o início da tarefa.

3 Implementação e modelo temporal

3.1 Arquitetura e operações simuladas

A aplicação foi organizada em quatro tarefas funcionais e duas auxiliares. A Figura 1 apresenta as filas e o estado compartilhado. A implementação utiliza os mecanismos de tarefas, filas e sincronização do FreeRTOS; os cálculos de fusão, PID e mistura de motores estão escritos no código da aplicação. O Python foi utilizado somente após as coletas, para organizar dados e produzir figuras.

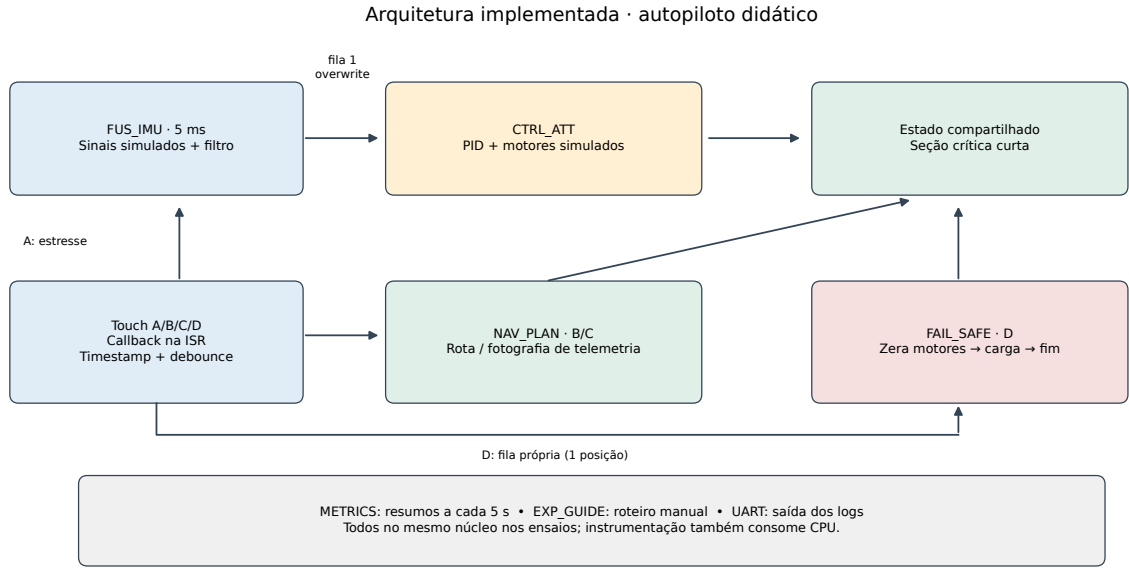


Figura 1: Arquitetura do autopiloto didático.

A medição de segurança termina após a carga artificial; a medição da telemetria termina antes da transmissão do texto.

Fonte: elaboração do autor a partir dos registros experimentais.

A `FUS_IMU` executa nominalmente a cada 5 ms, com espera baseada em `vTaskDelayUntil`. Sinais determinísticos simulam giroscópio e acelerômetro. Para roll e pitch, o filtro complementar combina a integração do giroscópio com a estimativa de aceleração:

$$\hat{\theta}_k = \alpha(\hat{\theta}_{k-1} + \omega_k \Delta t) + (1 - \alpha)\theta_{a,k}, \quad \alpha = 0,98, \quad \Delta t = 0,005 \text{ s.} \quad (1)$$

Yaw é integrado e limitado ao intervalo angular adotado. O estado contém número de sequência e timestamp de amostragem, sendo publicado por `xQueueOverwrite` em uma fila de uma posição.

A `CTRL_ATT` bloqueia nessa fila até receber um estado. O PID de cada eixo calcula a contribuição proporcional, integral e derivativa; a integral é limitada e a saída alimenta a mistura de quatro motores. Os comandos são restringidos entre 0% e 100%, com referência de sustentação simulada de 50%. O modelo permite exercitar o encadeamento temporal, mas não corresponde a uma validação de estabilidade de um drone físico.

A `NAV_PLAN` processa os eventos B e C em uma fila comum. B alterna o waypoint e atualiza referências de atitude. C obtém uma fotografia do estado e imprime sequência, atitude, motores e condições de estresse/segurança. Seções críticas curtas protegem as estruturas compartilhadas.

A `FAIL_SAFE`, equivalente à `FS_TASK` do enunciado, recebe D em uma fila própria, ativa uma condição segura persistente e zera os quatro motores simulados. O `CTRL` consulta essa condição antes de escrever nos atuadores, impedindo a reativação após a emergência. O fim instrumentado de FS ocorre depois de uma carga artificial adicional,

característica relevante para interpretar sua resposta.

As tarefas `METRICS` e `EXP_GUIDE`, de prioridade 1, emitem os resumos e orientam os toques. Em `UNICORE`, também executam no núcleo 0 e participam da carga global, embora não apareçam como categorias próprias nas quatro métricas principais.

3.2 Touch, interrupções e estresse

Foi utilizada a API `driver/touch_sens.h`, compatível com o ambiente instalado. A documentação e o exemplo oficial descrevem criação do controlador/canais, leitura e registro de callbacks [5, 6]. O mapeamento escolhido é mostrado na Tabela 1.

Tabela 1: Entradas touch utilizadas.

Entrada	Canal	Pino	Função
A	T0	GPIO4	Perturbação e carga adicional da fusão
B	T7	GPIO27	Comando de rota/waypoint
C	T4	GPIO13	Solicitação de telemetria
D	T9	GPIO32	Emergência e condição segura

Fonte: configuração da aplicação; correspondência canal/GPIO conforme documentação do ESP32 [6].

Após a calibração com os contatos livres, o limite de ativação corresponde a 80% da referência lida. A aplicação aceita um evento por contato, com debounce de 1.200 ms por canal e liberação estável de 500 ms. O callback registra o instante da detecção e encaminha o evento por operação apropriada à ISR. Assim, o trabalho extenso é realizado nas tarefas.

A carga de cálculo é reproduzida por `simulate_fixed_work`, com laço fixo e acumulador volátil. A Tabela 2 registra os parâmetros consultados. A carga tem quantidade de operações controlada; a duração depende da frequência e da interferência.

Tabela 2: Quantidade de trabalho artificial na versão consultada.

Trecho	Iterações	Condição
FUS	3.000	Cada job
CTRL	2.000	Cada estado consumido
NAV	24.000	Caminho de atualização de rota
FS	6.000	Após zerar motores
Estresse da FUS	500	A cada quatro amostras durante o estresse

Fonte: constantes de `main.c`.

O touch A perturba os sinais e mantém o estresse por 2.000 amostras, nominalmente 10 s a 200 Hz. Esse intervalo é contado por amostras, não por um temporizador independente. Os custos sugeridos pelo enunciado são referências a medir: não foram pressupostos como WCET real, nem se ajustou artificialmente o resultado medido a esses valores.

3.3 Períodos, deadlines e criticidade

Tabela 3: Modelo temporal adotado.

Tarefa	Ativação	Deadline	Classe
FUS	Periódica; T=5 ms	5 ms após liberação esperada	Hard
CTRL	Fila da FUS; taxa nominal até 200 Hz	5 ms desde a amostra	Hard
NAV	Eventos B/C	20 ms desde detecção na ISR	Soft
FS	Evento D via ISR	10 ms desde detecção na ISR	Hard
Auxiliares	Relatórios e guia	Sem deadline funcional de voo	Sem requisito crítico

Fonte: elaboração do autor conforme os requisitos e as referências temporais do firmware.

FUS e CTRL foram classificadas como hard porque mantêm a cadeia de atualização dos atuadores; atrasos comprometem o ciclo temporal pretendido. FS representa a segurança e também é hard. NAV foi classificada como soft, pois atrasar uma referência de alto nível ou uma consulta não equivale à interrupção imediata do controle local. A classificação descreve a consequência de perder um prazo; não é alterada para soft quando o ensaio apresenta falhas.

O timestamp usado por CTRL é capturado no início da FUS. Dessa forma, o deadline do controle abrange fusão, espera e processamento do controlador. As tarefas auxiliares não têm requisito funcional crítico, embora possam afetar a execução das demais.

3.4 Políticas e configuração do escalonador

Tabela 4: Prioridades implementadas; maior número indica maior prioridade.

Política	FUS	CTRL	FS	NAV	Critério
CUSTOM	4	3	5	2	Segurança, fusão, controle e navegação
DM	5	5	4	3	Deadlines crescentes: 5, 5, 10 e 20 ms
RM adaptado	5	5	4	3	Cadeia nominal de 200 Hz acima dos eventos; desempate por segurança

Fonte: seleção por `PRIORITY_POLICY` em `main.c`.

CUSTOM privilegia a emergência, aceitando que ela interfira no controle. DM protege primeiro os menores deadlines. FUS e CTRL empatam em DM/RM, tornando o time slicing potencialmente relevante quando ambas estão prontas. As prioridades

implementadas de RM e DM coincidem; por isso, diferenças entre suas execuções não representam uma ordenação diferente.

Para RM, o debounce por pad limita eventos individuais, mas NAV reúne B e C, que podem chegar próximos. Sua taxa agregada não possui intervalo mínimo de 1.200 ms. Trata-se de uma adaptação experimental de RM, sem uma demonstração teórica para chegadas agregadas e tarefas encadeadas.

O menuconfig selecionou UNICORE e frequência. Os valores de `configUSE_PREEMPTION` e `configUSE_TIME_SLICING` foram alterados na configuração do FreeRTOS para as coletas. O escalonamento por prioridades, o rodízio entre prioridades iguais e os pontos de bloqueio/yield são mecanismos distintos [7]. No caminho touch do SDK consultado, o callback pode solicitar `portYIELD_FROM_ISR` [8]; portanto, os ensaios com preempção zero são descritos como configuração cooperativa com yields explícitos, inclusive na saída da ISR.

4 Metodologia experimental e processamento

4.1 Matriz e protocolo

Foram analisados oito cenários da matriz principal (CUSTOM/DM, preemptivo/-cooperativo e 80/240 MHz), quatro adicionais a 160 MHz, dois DM preemptivos com slicing declarado desligado e quatro RM a 80/240 MHz. Cada combinação tem uma execução, com duração final de 30,000 a 35,015 s e operação em um núcleo confirmada no banner. Os IDs 01–08 identificam a matriz principal; E01–E04 correspondem a 160 MHz, E05–E06 ao slicing OFF e E07–E10 a RM.

O roteiro guiado aceita a sequência

C B C A B C B C B C B C D C.

São um toque A, cinco B, sete C e um D. Os registros incluem navegação e telemetria durante estresse e uma consulta após a emergência para verificar motores zerados e modo seguro. O contato é liberado entre acionamentos conforme o guia. O procedimento orienta reinicializar a placa antes de cada execução.

A validação dos registros verificou duração mínima, guia concluído, ordem e contagem dos eventos, zero descartes touch reportados, integridade das linhas METRIC e ausência das mensagens de watchdog e falhas de sequência pesquisadas. Foram extraídas 488 linhas, equivalentes a 122 conjuntos de quatro tarefas. Cada execução contém 12 jobs de NAV e um de FS.

O protocolo facilitou a repetição manual, mas não controla a fase exata do toque em relação à fusão. A sequência aceita também não demonstra um limite temporal entre B e C capaz de comprovar simultaneidade física. Essa limitação deve acompanhar a avaliação das rajadas previstas no enunciado.

4.2 Referências temporais e métricas

O firmware utiliza `esp_timer_get_time()`, que fornece uma referência de tempo em microssegundos [9]. Para um job k , sejam e_k , s_k e f_k os instantes de evento, início e fim. As grandezas implementadas são

$$E_k = \max(f_k - s_k, 0), \quad R_k = \max(f_k - e_k, 0), \quad (2)$$

$$L_k = \max(s_k - e_k, 0), \quad m_k = \mathbf{1}(R_k > D). \quad (3)$$

E é duração decorrida do trecho, R é resposta, L é latência de início e m_k indica perda do deadline. E pode conter interferência de outras tarefas; seu máximo observado não é um limite garantido de WCET de CPU.

FUS utiliza como evento uma referência esperada de liberação. CTRL utiliza o timestamp da amostra; NAV e FS usam a leitura feita no callback touch. Para estes últimos, a latência começa na detecção pela ISR, sem incluir o atraso físico anterior do periférico.

O jitter registrado é

$$J_k^{\text{FUS}} = |s_k - e_k|, \quad J_k^{\text{evento}} = |L_k - L_{k-1}|, \quad (4)$$

com o primeiro valor das tarefas por evento fixado em zero. São definições diferentes, mantidas nas tabelas e separadas nas figuras. Para FS, com um job, jitter zero não avalia repetibilidade.

4.3 Agregação, CPU e rastreabilidade

Jobs, misses, mínimos, médias e máximos são acumulados desde o início. Para as tabelas finais foi selecionado o último resumo completo de cada cenário. As taxas usam

$$p_{\text{miss}} = 100 \frac{N_{\text{miss}}}{N_{\text{jobs}}}. \quad (5)$$

A classe hard agrega FUS, CTRL e FS pela soma de perdas dividida pela soma de jobs. Esse valor é acompanhado da decomposição por tarefa, pois um miss no único FS pode ficar diluído entre milhares de jobs periódicos. NAV reúne os caminhos B e C.

A CPU do firmware deriva da diferença de runtime de cada tarefa entre consultas, dividida pela duração da janela. A média do ensaio foi aproximada por

$$\bar{U}_i = \frac{\sum_j U_{i,j} \Delta t_j}{\sum_j \Delta t_j}. \quad (6)$$

As janelas são obtidas dos instantes impressos. Consultas de runtime sequenciais e arredondamentos limitam a precisão. A soma das quatro tarefas não é CPU total; não se

identificou automaticamente o restante como IDLE.

Para a evolução temporal, foram calculadas diferenças dos contadores entre resumos. Os valores acumulados não foram somados entre si. Não se extrapolaram ensaios de 30 s para 35 s; tabelas apresentam duração e denominadores. Os gráficos e as tabelas foram gerados por Python/Matplotlib a partir dos mesmos dados derivados dos logs.

Foram preservados os registros originais, nomes de arquivos, linhas de origem e hashes SHA-256. As linhas de resumo armazenam os timestamps do último job de cada tarefa, não todos os timestamps individuais. Por isso não se calcularam percentis, histogramas de jobs ou intervalos de confiança a partir dessas médias. O material suplementar inclui os CSV completos e os 18 logs, permitindo conferir cada resultado.

5 Resultados e discussão

5.1 Comparação geral

A Tabela 5 reúne a configuração e os principais indicadores de todas as execuções. A Tabela 6 detalha os denominadores por tarefa, evitando que a taxa hard agregada oculte uma perda de segurança. Os tempos completos, incluindo mínimos, médias, máximos, jitter e CPU por tarefa, constam do Apêndice A.

Tabela 5: Comparação dos 18 cenários: configuração, perdas, segurança e CPU.

ID	Política	Modo	MHz	Slice	s	H (%)	S (%)	L_{FS}	R_{FS}	CPU (%)
01	CUSTOM	PRE	80	ON	35,000	0,007	41,667	0,020	4,259	69,863
02	CUSTOM	PRE	240	ON	35,000	0,000	0,000	0,009	1,348	23,210
03	CUSTOM	COOP	80	ON	35,015	0,208	41,667	0,021	4,799	69,811
04	CUSTOM	COOP	240	ON	35,012	0,136	0,000	0,008	1,347	23,183
05	DM	PRE	80	ON	35,000	0,007	41,667	0,024	14,792	69,816
06	DM	PRE	240	ON	35,000	0,000	0,000	0,013	3,162	23,195
07	DM	COOP	80	ON	35,015	0,521	41,667	1,420	9,042	69,976
08	DM	COOP	240	ON	35,012	0,114	0,000	0,013	1,351	23,182
E01	CUSTOM	PRE	160	ON	35,000	0,000	0,000	0,012	2,022	34,720
E02	CUSTOM	COOP	160	ON	35,013	0,179	0,000	0,024	2,047	34,708
E03	DM	PRE	160	ON	34,999	0,000	0,000	0,033	3,781	34,722
E04	DM	COOP	160	ON	35,013	0,186	0,000	0,017	2,026	34,761
E05	DM	PRE	80	OFF	35,003	0,007	41,667	3,517	18,983	69,795
E06	DM	PRE	240	OFF	30,001	0,000	0,000	0,013	1,351	23,262
E07	RM	PRE	240	ON	30,000	0,000	0,000	0,530	1,905	23,246
E08	RM	PRE	80	ON	30,000	0,008	41,667	0,948	16,871	69,971
E09	RM	COOP	80	ON	30,015	0,592	41,667	0,024	8,042	70,139
E10	RM	COOP	240	ON	35,012	0,114	0,000	0,025	1,367	23,192

Fonte: logs preservados; processamento do autor. PRE: preemptivo; COOP: configuração cooperativa. Todos UNICORE. Slicing declarado conforme histórico, não impresso no banner. Hard: FUS+CTRL+FS; soft: NAV (B+C). L e R de FS em ms; R termina após a carga artificial. CPU é a média aproximada da soma das quatro tarefas, não CPU total.

Tabela 6: Perdas por tarefa: misses/jobs e percentual acumulado.

ID	FUS	CTRL	NAV	FS
01	0/7000 (0,000%)	1/6999 (0,014%)	5/12 (41,667%)	0/1 (0,000%)
02	0/7000 (0,000%)	0/7000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
03	26/7001 (0,371%)	3/6967 (0,043%)	5/12 (41,667%)	0/1 (0,000%)
04	19/7001 (0,271%)	0/6974 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
05	0/7000 (0,000%)	0/7000 (0,000%)	5/12 (41,667%)	1/1 (100,000%)
06	0/7000 (0,000%)	0/7000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
07	48/7001 (0,686%)	25/7001 (0,357%)	5/12 (41,667%)	0/1 (0,000%)
08	16/7001 (0,229%)	0/7001 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E01	0/7000 (0,000%)	0/7000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E02	25/7001 (0,357%)	0/6975 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E03	0/7000 (0,000%)	0/7000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E04	26/7001 (0,371%)	0/7001 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E05	0/7000 (0,000%)	0/7000 (0,000%)	5/12 (41,667%)	1/1 (100,000%)
E06	0/6000 (0,000%)	0/6000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E07	0/6000 (0,000%)	0/6000 (0,000%)	0/12 (0,000%)	0/1 (0,000%)
E08	0/6000 (0,000%)	0/6000 (0,000%)	5/12 (41,667%)	1/1 (100,000%)
E09	47/6001 (0,783%)	24/6001 (0,400%)	5/12 (41,667%)	0/1 (0,000%)
E10	16/7001 (0,229%)	0/7001 (0,000%)	0/12 (0,000%)	0/1 (0,000%)

Fonte: logs preservados. Percentuais reportados pelo firmware. O contador da FUS depende da referência discutida na Seção 6.

Seis execuções não registraram perdas nas quatro tarefas: CUSTOM preemptivo a 160/240 MHz, DM preemptivo a 160/240 MHz, DM preemptivo a 240 MHz com slicing OFF e RM preemptivo a 240 MHz. Em todas as condições avaliadas a 80 MHz, NAV registrou 5/12 perdas (41,667%). Todos os ensaios configurados como cooperativos registraram perdas da FUS. A ausência de perdas em uma execução não demonstra uma garantia hard RT para todas as fases de entrada.

5.2 Prioridades: CUSTOM e DM

A 80 MHz no modo preemptivo, CUSTOM registrou uma perda do CTRL e nenhuma de FS; DM registrou zero perdas do CTRL e uma de FS. As respostas da emergência foram 4,259 ms e 14,792 ms, respectivamente, para D=10 ms. Esse comportamento é compatível com a prioridade máxima de FS em CUSTOM e com FUS/CTRL acima de FS em DM.

No CTRL de CUSTOM/80, a duração média foi 1,3561 ms, enquanto a máxima chegou a 10,165 ms e a resposta máxima a 12,223 ms. Em DM/80, a resposta máxima do CTRL foi 3,795 ms. Portanto, a média reduzida não elimina um atraso pontual importante. Sem trace de trocas de contexto, não é possível atribuir cada intervalo dessa duração a uma interferência específica.

A 240 MHz, CUSTOM e DM preemptivos registraram zero perdas em todas as

Matriz principal · perdas de deadline registradas
Verde: nenhuma reportada; vermelho: pelo menos uma

01 · CUSTOM PRE 80	0/7000 0,000%	1/6999 0,014%	5/12 41,667%	0/1 0,000%
02 · CUSTOM PRE 240	0/7000 0,000%	0/7000 0,000%	0/12 0,000%	0/1 0,000%
03 · CUSTOM COOP 80	26/7001 0,371%	3/6967 0,043%	5/12 41,667%	0/1 0,000%
04 · CUSTOM COOP 240	19/7001 0,271%	0/6974 0,000%	0/12 0,000%	0/1 0,000%
05 · DM PRE 80	0/7000 0,000%	0/7000 0,000%	5/12 41,667%	1/1 100,000%
06 · DM PRE 240	0/7000 0,000%	0/7000 0,000%	0/12 0,000%	0/1 0,000%
07 · DM COOP 80	48/7001 0,686%	25/7001 0,357%	5/12 41,667%	0/1 0,000%
08 · DM COOP 240	16/7001 0,229%	0/7001 0,000%	0/12 0,000%	0/1 0,000%
	FUS_IMU*	CTRL_ATT	NAV_PLAN (B+C)	FAIL_SAFE

Uma execução por cenário · *FUS: referência temporal com ressalva (ver auditoria)

Figura 2: Perdas de deadline por tarefa na matriz principal.

As células apresentam misses/jobs e percentual. As cores indicam ocorrência de perdas, sem equiparar sua gravidade entre tarefas. FUS utiliza a referência temporal discutida na Seção 6.

Fonte: elaboração do autor a partir dos registros experimentais.

tarefas. A resposta de FS foi 1,348 ms em CUSTOM e 3,162 ms em DM. A maior capacidade de processamento proporcionou folga observada para os deadlines nessa sequência. A Figura 3 apresenta os máximos e seus limites.

5.3 Resposta da emergência

A Figura 4 separa a latência até iniciar da duração do trecho. Como há um único job FS por execução, as duas parcelas pertencem ao mesmo evento e podem ser somadas. Em CUSTOM preemptivo, L foi de 20, 12 e 9 μ s para 80, 160 e 240 MHz, respectivamente; R foi de 4,259, 2,022 e 1,348 ms.

Em DM preemptivo/80, FS iniciou após 24 μ s, mas terminou 14,792 ms após o evento. A duração decorrida foi 14,768 ms. Assim, o miss não significa uma espera de quase 15 ms para começar: a tarefa começou cedo e o trecho completo terminou depois do prazo. Como FUS e CTRL têm maior prioridade, interferência durante esse trecho é uma explicação compatível com a implementação.

Os motores são escritos como zero antes da carga artificial, e esse instante não tem timestamp próprio. O resultado demonstra atraso de conclusão do job instrumentado, mas não permite afirmar que a ação de zerar os motores ocorreu somente ao final. Também não mede a latência anterior à detecção do touch. A observação 1/1 de FS descreve o evento coletado; não estima uma probabilidade de falha em operação prolongada.

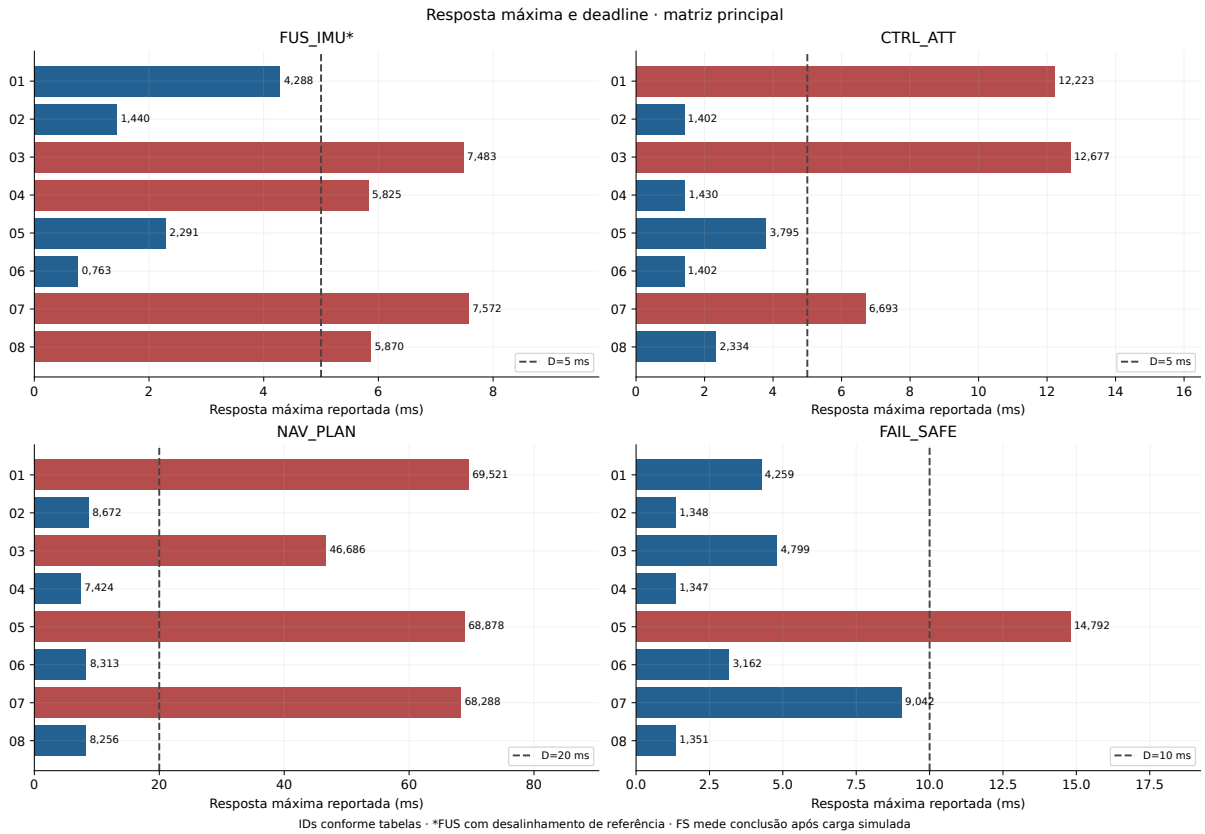


Figura 3: Respostas máximas e deadlines na matriz principal. As linhas tracejadas indicam os deadlines configurados. Os máximos são observados; em FS o fim é posterior à escrita dos motores zerados.

Fonte: elaboração do autor a partir dos registros experimentais.

5.4 Preempção, cooperação e jitter

Na matriz principal, CUSTOM cooperativo apresentou 26/7001 perdas de FUS a 80 MHz e 19/7001 a 240 MHz. DM cooperativo apresentou 48/7001 e 16/7001. O CTRL perdeu 3/6967 e 25/7001 jobs a 80 MHz, em CUSTOM e DM, respectivamente; a 240 MHz não registrou perdas.

Trechos longos sem preempção automática podem atrasar tarefas prontas até um ponto de bloqueio ou yield. Aumentar a frequência reduz o custo desses trechos, mas não substitui a análise dos pontos de reescalonamento. Além disso, a referência da FUS mistura ticks e microssegundos; por isso seus valores são apresentados como contadores reportados, com a ressalva da Seção 6.

O FS de DM cooperativo/80 concluiu em 9,042 ms, abaixo dos 14,792 ms do preemptivo. Essa observação não demonstra superioridade geral do cooperativo: a fase do evento varia e o caminho da ISR pode solicitar uma troca de tarefa. O trecho após o início também pode sofrer menos interrupções de tarefas superiores nessa configuração.

A Figura 5 mostra desvios máximos de início da FUS próximos de um período em vários cenários cooperativos. Os painéis distinguem a definição de jitter da FUS da

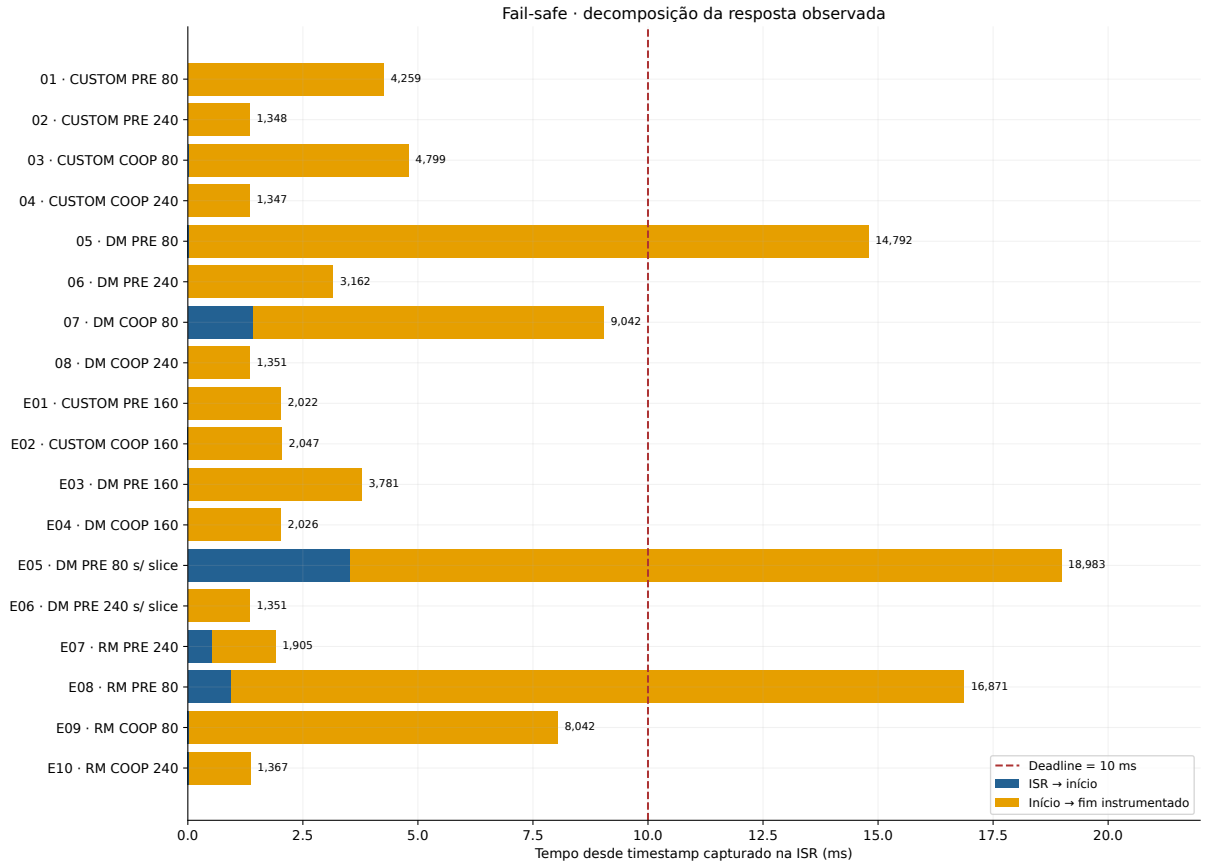


Figura 4: Decomposição da resposta de segurança nos 18 ensaios. L é o intervalo ISR–início; E é o intervalo início–fim instrumentado. A referência de 10 ms aplica-se ao deadline contado pelo firmware. Cada barra representa um evento de emergência.

Fonte: elaboração do autor a partir dos registros experimentais.

variação de latência entre jobs do CTRL. A Figura 6 apresenta a taxa de perdas por janela, obtida por diferenças de contadores acumulados. Ela localiza os aumentos em intervalos de aproximadamente 5 s, sem reconstruir o instante exato de cada perda ou isolar causalmente o efeito do touch A.

5.5 Frequência e extra de 160 MHz

Para CUSTOM preemptivo, o mínimo observado de E da FUS foi 2,027 ms a 80 MHz, 1,008 ms a 160 MHz e 0,668 ms a 240 MHz. No CTRL, os mínimos foram 1,352, 0,670 e 0,447 ms. A redução é compatível com um número fixo de cálculos executado em frequência maior. Esses mínimos caracterizam o custo base observado e não WCET.

O extra de 160 MHz acrescentou quatro execuções. CUSTOM e DM preemptivos não registraram perdas; os cooperativos registraram 25 e 26 perdas de FUS, respectivamente, sem perdas de CTRL, NAV ou FS. Assim, a frequência intermediária já eliminou, nas fases observadas, as perdas de NAV e dos preemptivos. Isso não estabelece 160 MHz como frequência mínima universalmente suficiente.

A soma média aproximada da CPU das quatro tarefas ficou em torno de 70% a

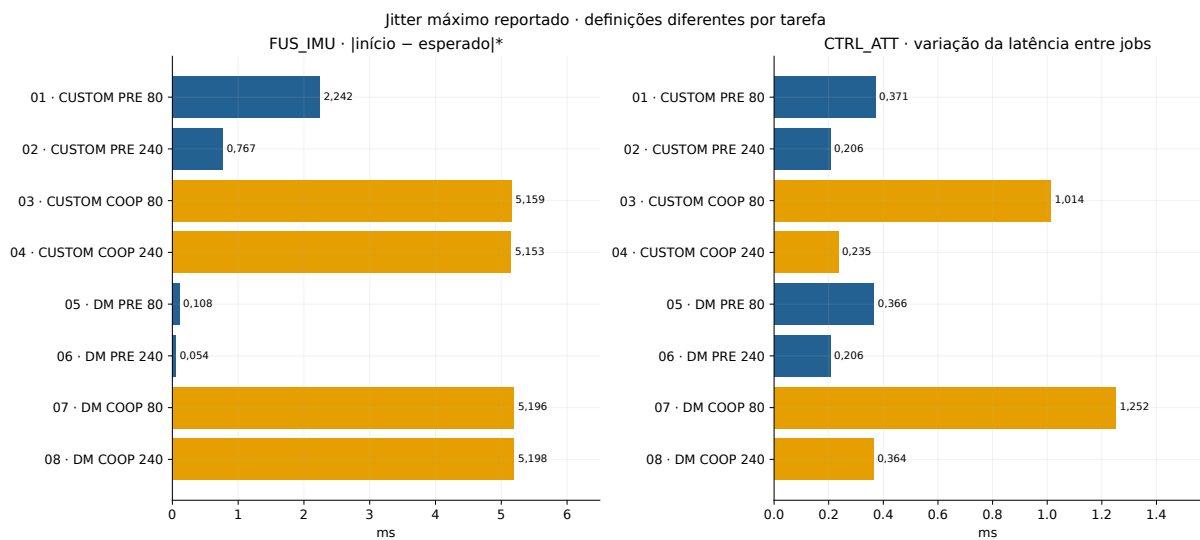


Figura 5: Jitter máximo reportado na matriz principal.
FUS: desvio absoluto da referência esperada, sujeito ao desalinhamento de fase. CTRL: diferença absoluta entre latências de jobs consecutivos. As duas estatísticas têm definições distintas.

Fonte: elaboração do autor a partir dos registros experimentais.

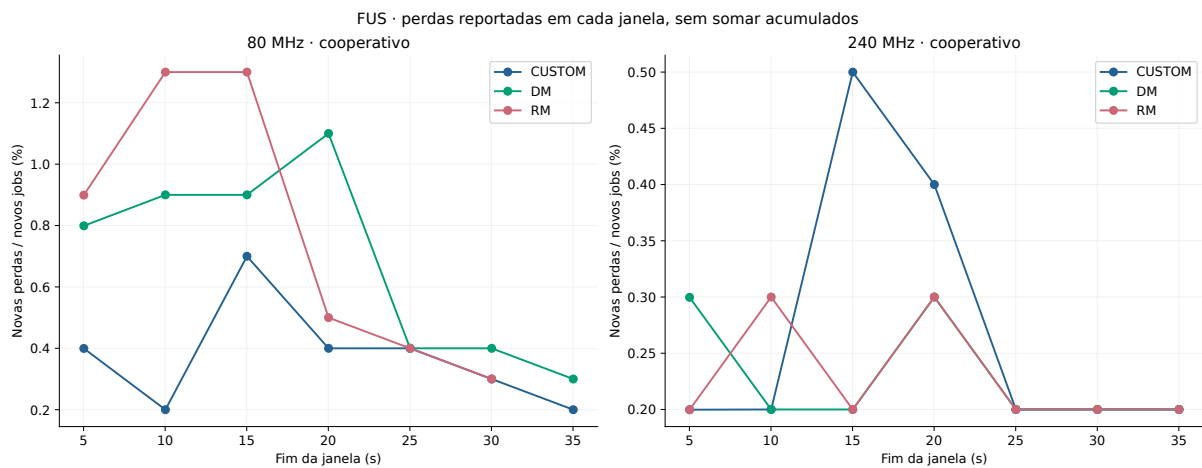


Figura 6: Evolução das perdas da FUS nas configurações cooperativas.
Cada série corresponde a uma execução. As taxas são por janela, não médias de repetições; fases dos eventos manuais diferem entre cenários.

Fonte: elaboração do autor a partir dos registros experimentais.

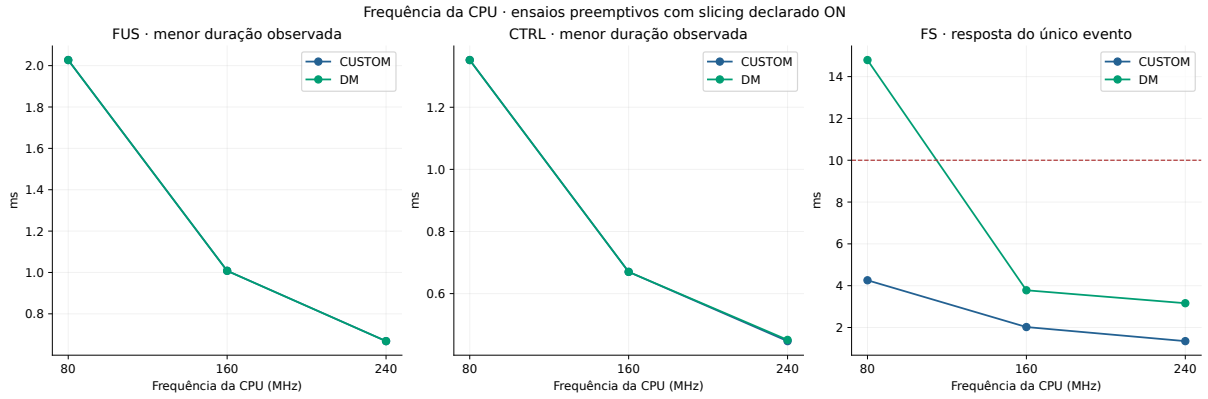


Figura 7: Efeito observado da frequência no custo base e na emergência. FUS/CTRL usam duração mínima observada; FS usa resposta do único evento. Linhas conectam medições, sem representar um modelo ajustado ou previsão para frequências não testadas.

Fonte: elaboração do autor a partir dos registros experimentais.

80 MHz, 35% a 160 MHz e 23% a 240 MHz (Figura 8). FUS e CTRL dominam o tempo acumulado devido à taxa nominal de 200 Hz. NAV tem baixa ocupação média, mas demanda concentrada em poucos eventos; sua pequena média não impede uma resposta máxima longa.

Na matriz principal, a resposta máxima de NAV variou de 46,686 a 69,521 ms a 80 MHz, acima de $D=20$ ms; a 240 MHz variou de 7,424 a 8,672 ms. O caminho de rota possui carga longa e prioridade inferior às tarefas críticas. A taxa 5/12 é da tarefa que reúne B e C; não pode ser automaticamente apresentada como taxa exclusiva de comandos de rota. O enunciado não fixa percentual tolerável para soft RT, portanto a taxa encontrada deve ser descrita como degradação de serviço, sem declará-la aceitável por definição.

Uma ocupação média abaixo de 100% não garante deadlines curtos sob demandas concentradas, bloqueios e prioridades fixas. Também não foi medida energia elétrica; os resultados de frequência e CPU não quantificam economia de energia.

5.6 Extra de time slicing

Os cenários E05/E06 repetem DM preemptivo a 80/240 MHz com slicing declarado OFF. FUS e CTRL têm a mesma prioridade, condição em que o rodízio por tick pode mudar sua alternância. Desligar slicing não elimina bloqueios, yields ou preempção por prioridades superiores [7].

A 80 MHz, a resposta de FS passou de 14,792 ms com ON para 18,983 ms com OFF; L passou de 0,024 para 3,517 ms. Ambos registraram um miss de FS e cinco de NAV, com zero perdas de FUS/CTRL. A 240 MHz, R de FS foi 3,162 ms com ON e 1,351 ms com OFF, sem perdas em ambas as execuções.

O sentido diferente dessa variação entre frequências impede concluir que desligar

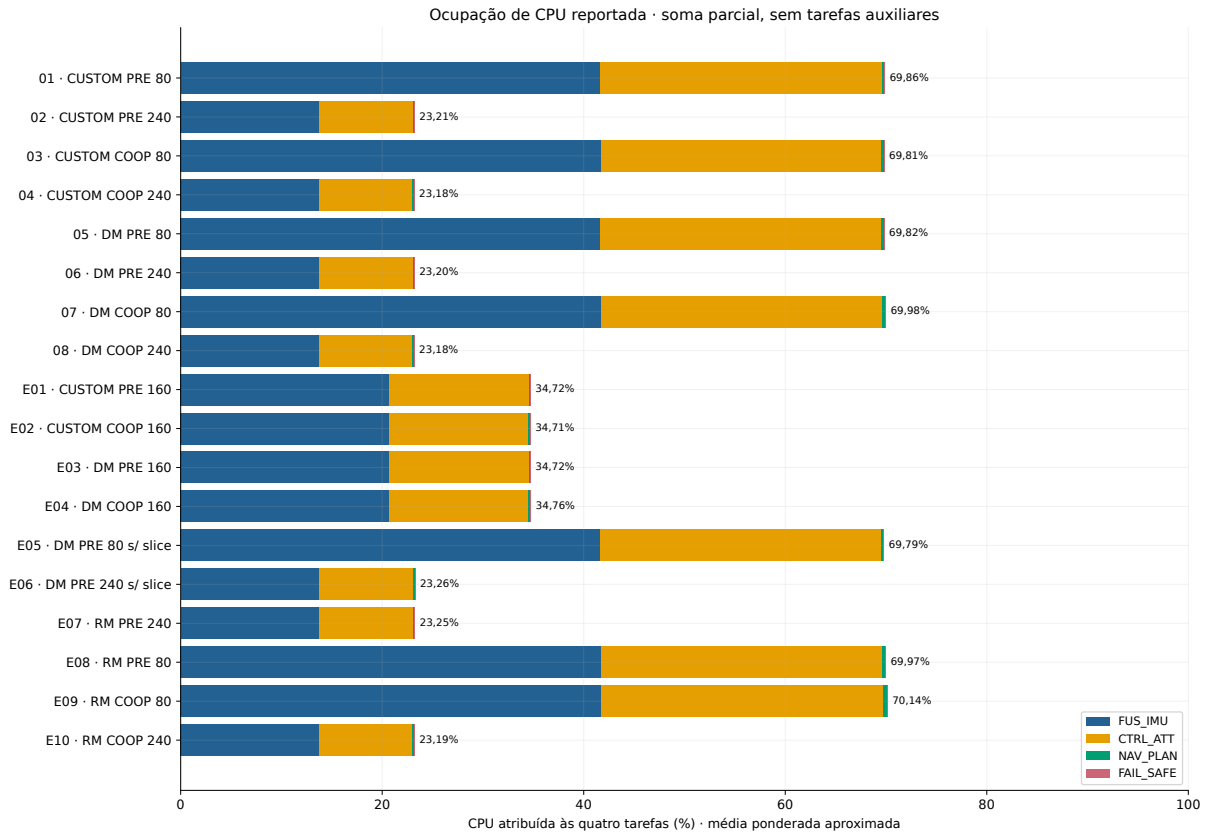


Figura 8: Ocupação média aproximada atribuída às quatro tarefas.

Média ponderada pelas janelas impressas. A soma exclui categorias próprias para tarefas auxiliares e sistema; seu complemento não foi interpretado como IDLE.

Fonte: elaboração do autor a partir dos registros experimentais.

slicing é sempre vantajoso ou prejudicial. Houve um evento por condição, sem sincronização de fase. O valor de slicing não foi impresso no banner e depende do histórico da coleta, limitando a confirmação retrospectiva.

5.7 Extra RM

RM preemptivo/240 registrou zero perdas; a 80 MHz, registrou 5/12 em NAV e 1/1 em FS, cuja resposta foi 16,871 ms. RM cooperativo/80 apresentou 47/6001 perdas de FUS e 24/6001 de CTRL; a 240 MHz, 16/7001 de FUS e zero nas demais tarefas. A proximidade com DM é compatível com a tabela de prioridades idêntica (Figura 10).

As durações diferentes tornam inadequada a comparação de contagens isoladas: DM cooperativo/80 teve 48 perdas de FUS em 7001 jobs, enquanto RM teve 47 em 6001. As taxas finais são 0,686% e 0,783%. Em resumos próximos de 30 s, os contadores foram 45/6001 e 47/6001, respectivamente. No cooperativo/240, DM e RM apresentaram 14/6001 nesse recorte. Não há evidência suficiente para atribuir superioridade a uma política, sobretudo porque sua ordenação implementada coincide.

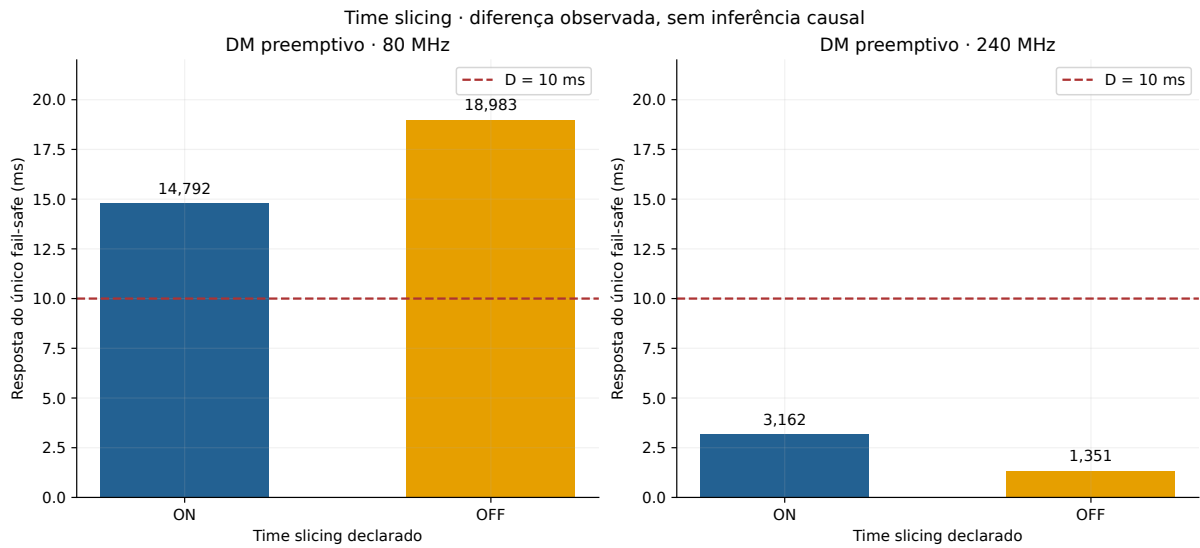


Figura 9: Comparação exploratória de time slicing em DM preemptivo. Os valores ON/OFF são declarados no histórico. As diferenças observadas não isolam o efeito causal do rodízio, devido à ausência de repetições e de controle de fase.

Fonte: elaboração do autor a partir dos registros experimentais.

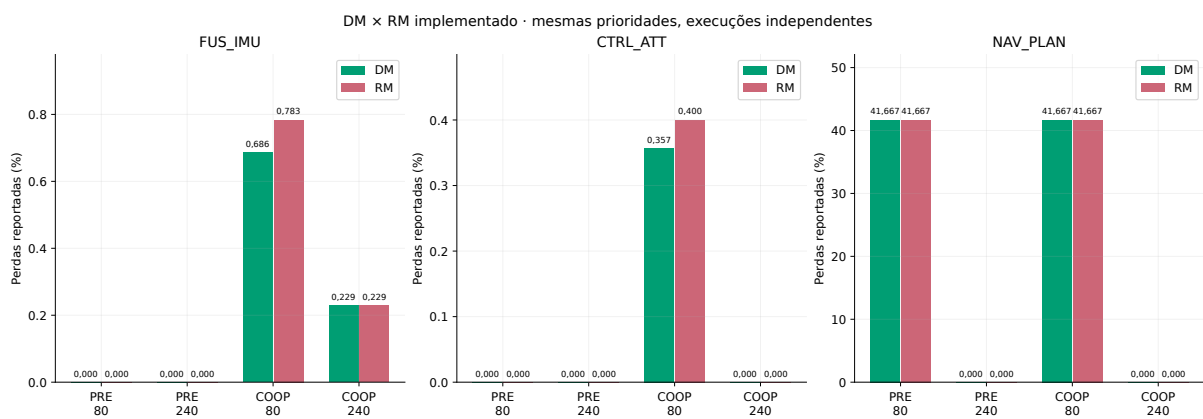


Figura 10: DM e RM: comparação das taxas reportadas. As prioridades implementadas são iguais. As taxas usam o denominador de cada execução; durações e fases dos eventos diferem. O tratamento agregado de B+C limita uma interpretação clássica de RM.

Fonte: elaboração do autor a partir dos registros experimentais.

5.8 Defasagem entre fusão e controle

Os snapshots finais de CUSTOM cooperativo apresentam diferenças de 34, 26 e 27 jobs entre FUS e CTRL, a 80, 160 e 240 MHz (Figura 11). A fila de uma posição mantém a amostra mais recente, podendo substituir estados ainda não consumidos. A diferença também pode incluir trabalho em trânsito no instante do resumo; não é um contador exato de amostras perdidas.

Os misses de CTRL são contados sobre jobs concluídos. Portanto, zero misses não demonstra consumo de todas as amostras. Seria necessário registrar os saltos de sequência e o trabalho pendente para separar as parcelas. O campo de descartes touch pertence a outro caminho de eventos e não cobre essa fila.

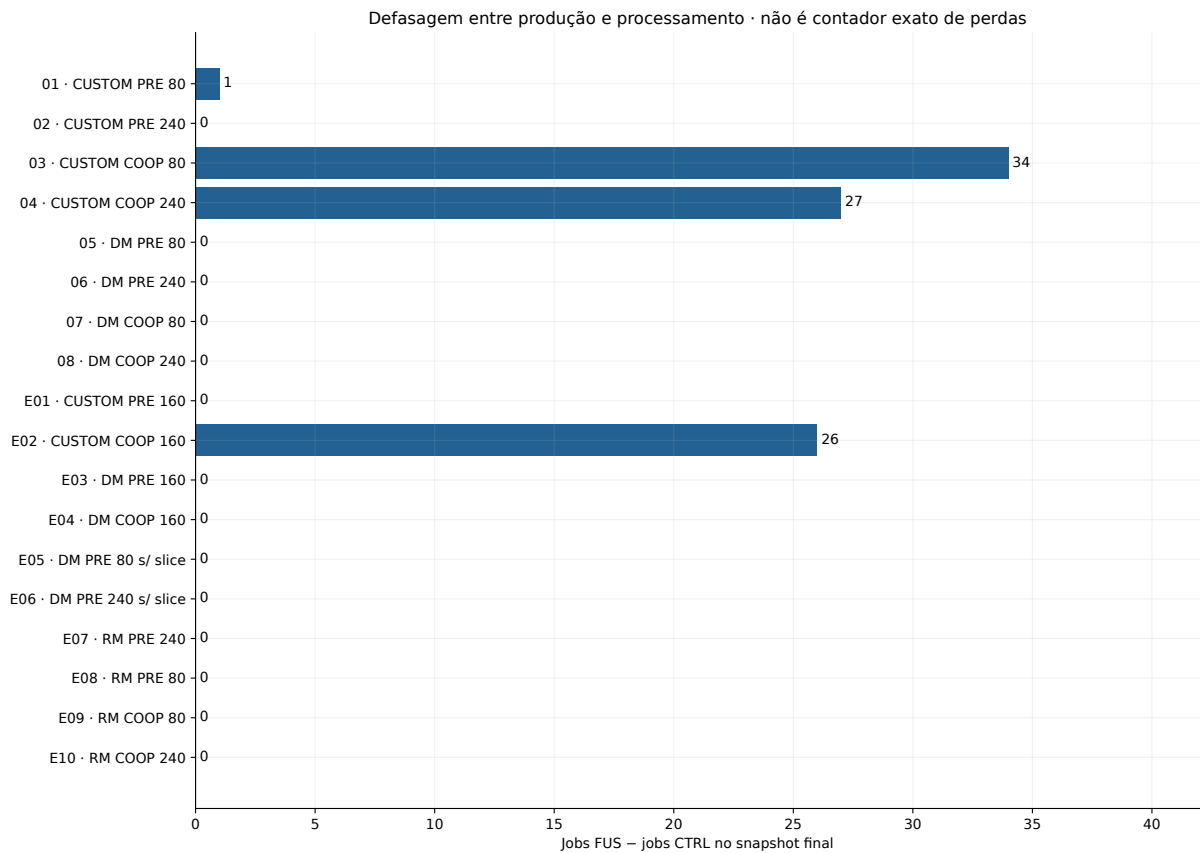


Figura 11: Diferença de jobs entre FUS e CTRL no snapshot final. Indicador de defasagem, que pode incluir sobrescrita e trabalho em trânsito. Não representa uma contagem exata de amostras descartadas.

Fonte: elaboração do autor a partir dos registros experimentais.

6 Limitações e atendimento aos requisitos

6.1 Consistência da instrumentação

A auditoria identificou um desalinhamento de referência na FUS: a espera periódica usa ticks, enquanto o instante esperado em microssegundos parte de uma leitura independente. Há snapshots em que o início real precede o evento esperado (Figura 12). No último snapshot do cenário 01, evento, início e fim são 35.042.128, 35.042.024 e 35.044.052 μs . A duração desse job é 2.028 μs e a resposta calculada é 1.924 μs ; a diferença decorre de o início anteceder a referência em 104 μs .

Como L é truncada em zero, não vale necessariamente $R = L + E$ nesse caso. O offset afeta latência, jitter e classificação de jobs próximos do deadline. As tabelas mantêm os resultados reportados. Não é possível corrigir retrospectivamente todos os máximos e misses com os timestamps do último job de cada resumo.

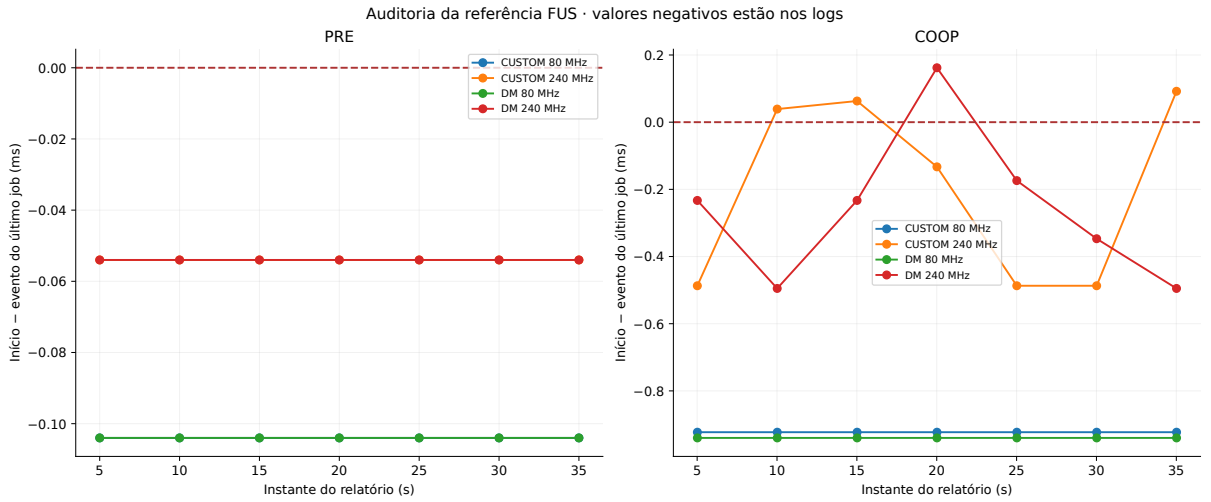


Figura 12: Auditoria da referência temporal da FUS.

Cada ponto representa somente o último job do resumo. Valores negativos indicam início anterior ao evento esperado; nenhuma correção foi aplicada aos dados.

Fonte: elaboração do autor a partir dos registros experimentais.

Outras limitações relevantes são a duração E com interferência, a medição de FS posterior à ação de zerar motores e a agregação dos dois tipos de evento NAV. No caminho C, a medida termina antes do `printf`; portanto não demonstra o prazo de conclusão da transmissão UART. Nas tarefas por evento, o primeiro jitter é zero e participa do denominador da média.

O modo cooperativo conserva os yields explícitos solicitados pelo caminho de interrupção. A documentação de escalonamento fornece a base conceitual [7], mas a interpretação concreta depende também do driver consultado [8]. Não se presume execução cooperativa pura até o término de cada job.

Por fim, há uma execução por combinação e um evento FS por execução, ausência

de controle preciso da fase dos toques e falta de um hash do binário em cada log. A consulta ao código atual auxilia a interpretação, mas não comprova a identidade de todos os binários históricos. Os campos de CPU são aproximados e não constituem perfil completo de ocupação do núcleo.

6.2 Cobertura do enunciado

Tabela 7: Síntese do atendimento técnico ao enunciado.

Requisito	Evidência e alcance
Quatro tarefas em C, touch e ISR	Fusão, PID, rota/telemetria e segurança implementados; sinais inerciais e atuadores simulados.
Modelo hard/soft e prioridades	Classificação justificada; DM e CUSTOM avaliados na matriz principal.
UNICORE, preempção e 80/240 MHz	Oito combinações obrigatórias coletadas; banners confirmam modo, núcleo e frequência.
Ensaio de pelo menos 30 s	Todas as execuções atingiram o mínimo e concluíram o guia. Proximidade/simultaneidade física não quantificada.
Timestamps, perdas, latência e CPU	Instrumentação e 488 linhas METRIC; persistem acumuladores e últimos timestamps por janela.
WCET e jitter	Máximos observados e jitter reportado; ressalvas sobre interferência e fase da FUS.
Todos os deadlines atendidos	Não demonstrado globalmente: há violações registradas e cenários sem perdas.
Extras RM, 160 MHz e slicing	Quatro, quatro e dois ensaios adicionais, respectivamente; resultados e limitações discutidos.
Tabelas, figuras e técnicas	Comparação dos 18 cenários, 12 figuras e métricas completas nos apêndices.

Fonte: confronto entre enunciado, implementação e registros.

A conclusão do roteiro confirma a execução da coleta, mas não equivale ao atendimento de todos os deadlines. O critério de sucesso do enunciado exige reconhecer as perdas observadas, inclusive quando a tabela ilustrativa do próprio PDF usa uma conclusão positiva ao lado de percentuais hard diferentes de zero.

Uma nova rodada mais rigorosa deveria alinhar a referência temporal da FUS, registrar o instante da escrita segura, contar sequências consumidas pelo CTRL, distinguir tempo de CPU de duração decorrida e repetir eventos em diferentes fases. Também seria útil imprimir slicing, parâmetros de carga e hash do firmware em cada execução. Esses aprimoramentos não foram aplicados retrospectivamente às coletas deste relatório.

7 Conclusão

O autopiloto didático permitiu exercitar uma cadeia periódica de fusão e controle em conjunto com navegação, telemetria e emergência acionadas por touch. A bateria de 18 ensaios cobriu a matriz principal e os extras de RM, 160 MHz e time slicing, mantendo a execução em um núcleo e duração mínima de 30 s.

Os resultados indicam que aumentar a frequência reduziu o custo base e a ocupação aproximada das tarefas. CUSTOM e DM preemptivos a 160 e 240 MHz não registraram perdas nesta bateria. A 80 MHz, NAV perdeu prazos em todas as condições avaliadas, enquanto CUSTOM favoreceu a conclusão de FS e DM protegeu a cadeia com deadline de 5 ms. Essa diferença expressa a escolha entre prioridade por criticidade e por prazo relativo.

Os cenários cooperativos registraram perdas da FUS mesmo a 240 MHz, mostrando que capacidade de processamento, pontos de reescalonamento e referência de liberação precisam ser analisados conjuntamente. RM apresentou comportamento próximo de DM, coerente com suas prioridades implementadas iguais. A variação de slicing forneceu observações adicionais, mas não isolou seu efeito devido ao número reduzido de eventos e à fase manual dos toques.

A análise também delimitou o alcance das medições: máximos de duração com interferência não são WCET garantido, a resposta completa de FS não identifica o instante da ação segura e a referência da FUS afeta seus indicadores. Assim, o trabalho apresenta evidências experimentais de atendimento e descumprimento de prazos, sem demonstrar garantia temporal para qualquer cenário admissível. Esses resultados orientam tanto a escolha da configuração quanto os próximos ajustes de instrumentação e repetição experimental.

Referências

- [1] Felipe Viel. *Avaliação M1 – Modelagem de Sistema: Sistemas em Tempo Real*. Universidade do Vale do Itajaí, 2026. Enunciado fornecido na disciplina, 4 p. Cópia no material suplementar: fontes/enunciado.pdf.
- [2] Espressif Systems. ESP-IDF: Espressif iot development framework, s.d.. URL <https://github.com/espressif/esp-idf>. Ambiente de desenvolvimento identificado como ESP-IDF v6.1. Acesso em: 23 set. 2026.
- [3] Espressif Systems. *ESP-IDF Programming Guide: FreeRTOS (IDF)*, s.d.. URL https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos_idf.html. Documentação oficial, versão online latest. Acesso em: 23 set. 2026.
- [4] Felipe Viel. Drone_exemplo_oTask.c: código de referência para ESP32 e FreeRTOS. Repositório Real_Time_Systems, GitHub, s.d. URL https://github.com/VielF/Real_Time_Systems/blob/main/Assignments/Drone_exemplo_oTask.c. Referência indicada na disciplina; cópia local consultada em 23 set. 2026.
- [5] Espressif Systems. Capacity touch sensor example: touch_sens_basic, s.d.. URL https://github.com/espressif/esp-idf/tree/fff9895c82d744c7237be8847347bdd1b07c6643/examples/peripherals/touch_sensor/touch_sens_basic. Exemplo oficial consultado também no SDK local. Revisão fff9895c82d744c7237be8847347bdd1b07c6643. Consulta em: 23 set. 2026.
- [6] Espressif Systems. *ESP-IDF Programming Guide: Capacitive Touch Sensor*, s.d.. URL https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/cap_touch_sens.html. Documentação oficial para ESP32. Acesso em: 23 set. 2026.
- [7] Richard Barry. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 161204 edition, 2016. URL https://www.freertos.org/media/2018/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf. Fundamentos de tarefas, filas e escalonamento. Acesso em: 23 set. 2026.
- [8] Espressif Systems. Driver touch v1: touch_version_specific.c, s.d.. URL https://github.com/espressif/esp-idf/blob/fff9895c82d744c7237be8847347bdd1b07c6643/components/esp_driver_touch_sens/hw_ver1/touch_version_specific.c. Código do SDK local consultado; revisão fff9895c82d744c7237be8847347bdd1b07c6643. Consulta em: 23 set. 2026.
- [9] Espressif Systems. *ESP-IDF Programming Guide: ESP Timer (High Resolution Timer)*, s.d.. URL https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_timer.html. Documentação oficial. Acesso em: 23 set. 2026.

A Métricas completas por tarefa

As Tabelas 8–11 reúnem os 72 resumos finais. E é duração decorrida, R é resposta, L é latência de início e J segue a definição de cada tarefa. Tempos em ms; CPU média ponderada. Deadlines: FUS/CTRL=5 ms, NAV=20 ms e FS=10 ms. Configuração e duração dos IDs: Tabela 5. As médias temporais preservam a precisão de 0,1 μ s impressa no log.

Tabela 8: Métricas finais de FUS_IMU: tempos em ms e CPU média em porcentagem.

ID	Jobs	Misses	%	$E_{\min}$	$E_{\text{méd}}$	$E_{\max}$	$R_{\min}$	$R_{\text{méd}}$	$R_{\max}$	$L_{\max}$	$J_{\text{méd}}$	$J_{\max}$	CPU
01	7000	0	0,000	2,027	2,0543	2,395	1,921	1,9507	4,288	2,242	0,1042	2,242	41,657
02	7000	0	0,000	0,668	0,6774	0,794	0,615	0,6235	1,440	0,767	0,0541	0,767	13,746
03	7001	26	0,371	2,021	2,2485	3,070	1,103	1,3627	7,483	5,159	0,9348	5,159	41,730
04	7001	19	0,271	0,666	0,6768	0,995	0,179	0,3072	5,825	5,153	0,4311	5,153	13,730
05	7000	0	0,000	2,027	2,0527	2,395	1,920	1,9488	2,291	0,005	0,1040	0,108	41,631
06	7000	0	0,000	0,668	0,6772	0,797	0,614	0,6232	0,763	0,000	0,0540	0,054	13,734
07	7001	48	0,686	2,024	2,2478	3,069	1,087	1,3648	7,572	5,196	0,9611	5,196	41,715
08	7001	16	0,229	0,667	0,6768	0,790	0,171	0,2998	5,870	5,198	0,4358	5,198	13,727
E01	7000	0	0,000	1,008	1,0205	1,187	0,938	0,9505	1,117	0,000	0,0700	0,071	20,699
E02	7001	25	0,357	1,005	1,0203	1,404	0,399	0,5630	6,181	5,173	0,5172	5,173	20,693
E03	7000	0	0,000	1,008	1,0216	1,188	0,937	0,9506	1,117	0,000	0,0710	0,071	20,700
E04	7001	26	0,371	1,007	1,0203	1,291	0,418	0,5883	6,150	5,100	0,5033	5,100	20,689
E05	7000	0	0,000	2,027	2,0529	2,402	1,682	1,7140	2,162	0,005	0,3389	0,345	41,614
E06	6000	0	0,000	0,668	0,6779	0,797	0,485	0,4950	0,763	0,000	0,1830	0,183	13,769
E07	6000	0	0,000	0,668	0,6778	0,790	0,614	0,6239	0,763	0,000	0,0540	0,054	13,761
E08	6000	0	0,000	2,027	2,0570	2,399	1,923	1,9531	2,295	0,005	0,1039	0,107	41,722
E09	6001	47	0,783	2,024	2,2494	3,091	1,091	1,3803	7,408	5,138	0,9621	5,138	41,795
E10	7001	16	0,229	0,668	0,6767	0,790	0,175	0,3044	5,864	5,192	0,4309	5,192	13,725

Fonte: resumos finais dos logs. E inclui interferência; seu máximo observado não é WCET garantido.

Tabela 9: Métricas finais de CTRL_ATT: tempos em ms e CPU média em porcentagem.

ID	Jobs	Misses	%	$E_{\min}$	$E_{\text{méd}}$	$E_{\max}$	$R_{\min}$	$R_{\text{méd}}$	$R_{\max}$	$L_{\max}$	$J_{\text{méd}}$	$J_{\max}$	CPU
01	6999	1	0,014	1,352	1,3561	10,165	3,410	3,4410	12,223	2,429	0,0494	0,371	27,910
02	7000	0	0,000	0,447	0,4513	0,513	1,129	1,1396	1,402	0,889	0,0179	0,206	9,307
03	6967	3	0,043	1,352	1,3567	10,620	3,404	3,6374	12,677	3,101	0,3802	1,014	27,771
04	6974	0	0,000	0,447	0,4506	0,514	1,127	1,1379	1,430	0,916	0,0176	0,235	9,249
05	7000	0	0,000	1,352	1,3534	1,419	3,410	3,4370	3,795	2,425	0,0488	0,366	27,884
06	7000	0	0,000	0,451	0,4513	0,513	1,129	1,1390	1,402	0,889	0,0171	0,206	9,306
07	7001	25	0,357	1,349	1,3640	4,445	3,406	3,6454	6,693	3,339	0,3811	1,252	27,916
08	7001	0	0,000	0,446	0,4511	1,652	1,128	1,1386	2,334	1,042	0,0180	0,364	9,289
E01	7000	0	0,000	0,670	0,6713	0,739	1,694	1,7073	1,973	1,234	0,0249	0,206	13,829
E02	6975	0	0,000	0,669	0,6721	2,536	1,691	1,7085	3,563	1,555	0,0250	0,531	13,800
E03	7000	0	0,000	0,670	0,6712	0,739	1,694	1,7082	1,973	1,234	0,0249	0,206	13,830
E04	7001	0	0,000	0,669	0,6729	2,642	1,692	1,7088	3,665	1,655	0,0252	0,631	13,851
E05	7000	0	0,000	1,352	1,3535	1,430	3,409	3,4373	3,803	2,433	0,0493	0,375	27,880
E06	6000	0	0,000	0,450	0,4514	0,510	1,129	1,1408	1,403	0,893	0,0203	0,210	9,306
E07	6000	0	0,000	0,451	0,4513	0,513	1,129	1,1402	1,402	0,889	0,0195	0,206	9,305
E08	6000	0	0,000	1,352	1,3541	1,419	3,410	3,4422	3,806	2,434	0,0568	0,367	27,900
E09	6001	24	0,400	1,349	1,3651	4,493	3,406	3,6486	6,780	3,392	0,3767	1,335	27,918
E10	7001	0	0,000	0,446	0,4509	1,957	1,128	1,1383	2,651	0,897	0,0177	0,211	9,291

Fonte: último resumo completo de cada execução. E: duração decorrida; R: resposta; L: latência; J: jitter conforme a definição de cada tarefa. Médias temporais mantêm a precisão de $0,1 \mu\text{s}$ impressa no log. Os IDs remetem à Tabela 5. Máximos observados não são WCET garantido. CPU média aproximada por ponderação das janelas.

Tabela 10: Métricas finais de NAV_PLAN: tempos em ms e CPU média em porcentagem.

ID	Jobs	Misses	%	$E_{\min}$	$E_{\text{méd}}$	$E_{\max}$	$R_{\min}$	$R_{\text{méd}}$	$R_{\max}$	$L_{\max}$	$J_{\text{méd}}$	$J_{\max}$	CPU
01	12	5	41,667	0,004	27,2887	69,463	0,028	28,4220	69,521	3,190	1,3316	2,616	0,283
02	12	0	0,000	0,001	3,2092	8,662	0,011	3,3405	8,672	1,148	0,2294	1,138	0,152
03	12	5	41,667	0,008	18,1842	46,663	0,038	19,7469	46,686	3,420	1,9231	3,397	0,297
04	12	0	0,000	0,005	2,5229	7,414	0,015	2,5415	7,424	0,065	0,0135	0,056	0,200
05	12	5	41,667	0,004	26,0068	65,910	0,063	27,7687	68,878	3,079	1,2426	2,560	0,289
06	12	0	0,000	0,001	3,0454	8,300	0,015	3,1902	8,313	0,925	0,2467	0,899	0,151
07	12	5	41,667	0,008	23,6338	65,761	1,039	25,7528	68,288	3,586	1,5258	3,392	0,333
08	12	0	0,000	0,005	2,6051	8,243	0,015	2,6870	8,256	0,397	0,1321	0,384	0,162
E01	12	0	0,000	0,002	5,4647	14,442	0,015	5,7783	14,456	1,630	0,3258	1,009	0,186
E02	12	0	0,000	0,006	4,0774	11,543	0,020	5,0316	12,537	1,771	0,5478	1,613	0,208
E03	12	0	0,000	0,002	5,1583	13,490	0,214	5,8666	14,008	1,621	0,4992	1,597	0,185
E04	12	0	0,000	0,006	4,6625	14,911	0,020	5,2795	14,928	1,771	0,9292	1,754	0,215
E05	12	5	41,667	0,004	26,8557	70,115	0,030	28,3203	73,472	3,357	0,9261	2,735	0,288
E06	12	0	0,000	0,001	2,9216	7,434	0,011	3,1598	7,986	0,841	0,4447	0,827	0,181
E07	12	0	0,000	0,001	2,9976	8,632	0,011	3,0912	8,641	0,928	0,1492	0,914	0,175
E08	12	5	41,667	0,004	26,8484	66,234	0,101	28,1378	68,804	2,976	1,1562	2,921	0,334
E09	12	5	41,667	0,008	25,2097	66,134	0,031	26,6322	68,613	3,411	1,4993	3,386	0,411
E10	12	0	0,000	0,005	2,5365	7,241	0,015	2,7185	7,254	1,222	0,1673	0,814	0,171

Fonte: último resumo completo de cada execução. E: duração decorrida; R: resposta; L: latência; J: jitter conforme a definição de cada tarefa. Médias temporais mantêm a precisão de $0,1 \mu\text{s}$ impressa no log. Os IDs remetem à Tabela 5. Máximos observados não são WCET garantido. CPU média aproximada por ponderação das janelas.

Tabela 11: Métricas finais de FAIL_SAFE: tempos em ms e CPU média em porcentagem.

ID	Jobs	Misses	%	$E_{\min}$	$E_{\text{méd}}$	$E_{\max}$	$R_{\min}$	$R_{\text{méd}}$	$R_{\max}$	$L_{\max}$	$J_{\text{méd}}$	$J_{\max}$	CPU
01	1	0	0,000	4,239	4,2390	4,239	4,259	4,2590	4,259	0,020	0,0000	0,000	0,013
02	1	0	0,000	1,339	1,3390	1,339	1,348	1,3480	1,348	0,009	0,0000	0,000	0,004
03	1	0	0,000	4,778	4,7780	4,778	4,799	4,7990	4,799	0,021	0,0000	0,000	0,013
04	1	0	0,000	1,339	1,3390	1,339	1,347	1,3470	1,347	0,008	0,0000	0,000	0,005
05	1	1	100,000	14,768	14,7680	14,768	14,792	14,7920	14,792	0,024	0,0000	0,000	0,013
06	1	0	0,000	3,149	3,1490	3,149	3,162	3,1620	3,162	0,013	0,0000	0,000	0,005
07	1	0	0,000	7,622	7,6220	7,622	9,042	9,0420	9,042	1,420	0,0000	0,000	0,013
08	1	0	0,000	1,338	1,3380	1,338	1,351	1,3510	1,351	0,013	0,0000	0,000	0,004
E01	1	0	0,000	2,010	2,0100	2,010	2,022	2,0220	2,022	0,012	0,0000	0,000	0,007
E02	1	0	0,000	2,023	2,0230	2,023	2,047	2,0470	2,047	0,024	0,0000	0,000	0,007
E03	1	0	0,000	3,748	3,7480	3,748	3,781	3,7810	3,781	0,033	0,0000	0,000	0,007
E04	1	0	0,000	2,009	2,0090	2,009	2,026	2,0260	2,026	0,017	0,0000	0,000	0,006
E05	1	1	100,000	15,466	15,4660	15,466	18,983	18,9830	18,983	3,517	0,0000	0,000	0,013
E06	1	0	0,000	1,338	1,3380	1,338	1,351	1,3510	1,351	0,013	0,0000	0,000	0,005
E07	1	0	0,000	1,375	1,3750	1,375	1,905	1,9050	1,905	0,530	0,0000	0,000	0,005
E08	1	1	100,000	15,923	15,9230	15,923	16,871	16,8710	16,871	0,948	0,0000	0,000	0,015
E09	1	0	0,000	8,018	8,0180	8,018	8,042	8,0420	8,042	0,024	0,0000	0,000	0,015
E10	1	0	0,000	1,342	1,3420	1,342	1,367	1,3670	1,367	0,025	0,0000	0,000	0,004

Fonte: último resumo completo de cada execução. E: duração decorrida; R: resposta; L: latência; J: jitter conforme a definição de cada tarefa. Médias temporais mantêm a precisão de $0,1 \mu\text{s}$ impressa no log. Os IDs remetem à Tabela 5. Máximos observados não são WCET garantido. CPU média aproximada por ponderação das janelas.

A Tabela 12 preserva os timestamps brutos do último job e a CPU da última janela, distinta da média do ensaio. Os CSV suplementares mantêm todas as 488 linhas de resumo e as diferenças por janela.

Tabela 12: Timestamps do último job e CPU da última janela por cenário/tarefa.

ID	Tarefa	Evento (μ s)	Início (μ s)	Fim (μ s)	CPU últ. (%)
01	FUS	35042128	35042024	35044052	41,128
01	CTRL	35042024	35044082	35045435	27,880
01	NAV	24534958	24535501	24535505	0,000
01	FS	23019744	23019764	23024003	0,000
02	FUS	35040621	35040567	35041236	13,576
02	CTRL	35040567	35041246	35041697	9,300
02	NAV	22109162	22109172	22109173	0,000
02	FS	20444906	20444915	20446254	0,000
03	FUS	35059938	35059015	35061853	41,233
03	CTRL	35059015	35061888	35063254	27,837
03	NAV	25909849	25913264	25913272	0,000
03	FS	24597092	24597113	24601891	0,000
04	FUS	35056043	35056135	35056807	13,566
04	CTRL	35056135	35056818	35057265	9,255
04	NAV	24364069	24364078	24364084	0,000
04	FS	22863822	22863830	22865169	0,000
05	FUS	35042136	35042032	35044060	41,151
05	CTRL	35042032	35044090	35045443	27,862
05	NAV	32391468	32391526	32391531	0,042
05	FS	30695761	30695785	30710553	0,090
06	FUS	35040629	35040575	35041244	13,568
06	CTRL	35040575	35041258	35041709	9,302
06	NAV	26762381	26762395	26762396	0,000
06	FS	24855517	24855530	24858679	0,000
07	FUS	35059962	35059022	35061863	41,222
07	CTRL	35059022	35061897	35063263	27,912
07	NAV	24970969	24972982	24972990	0,000
07	FS	23626086	23627506	23635128	0,000
08	FUS	35056059	35055564	35056232	13,564
08	CTRL	35055564	35056246	35056698	9,290
08	NAV	26210332	26210357	26210363	0,000
08	FS	24747799	24747812	24749150	0,000
E01	FUS	35041755	35041685	35042693	20,448
E01	CTRL	35041685	35042708	35043379	13,820
E01	NAV	28613800	28613814	28613816	0,000
E01	FS	27014167	27014179	27016189	0,000
E02	FUS	35056279	35055673	35056681	20,455
E02	CTRL	35055673	35056697	35057367	13,819
E02	NAV	32500084	32500098	32500104	0,089
E02	FS	30982457	30982481	30984504	0,044

Continua na próxima página.

Tabela 12 – continuação

ID	Tarefa	Evento (μs)	Início (μs)	Fim (μs)	CPU últ. (%)
E03	FUS	35041763	35041692	35042702	20,450
E03	CTRL	35041692	35042718	35043388	13,822
E03	NAV	26818056	26818424	26818427	0,000
E03	FS	25380198	25380231	25383979	0,000
E04	FUS	35056268	35055692	35056700	20,449
E04	CTRL	35055692	35056716	35057386	13,845
E04	NAV	28419616	28419629	28419636	0,000
E04	FS	26953016	26953033	26955042	0,000
E05	FUS	35045374	35045031	35047061	41,116
E05	CTRL	35045031	35047092	35048444	27,866
E05	NAV	25759887	25759913	25759917	0,000
E05	FS	24630003	24633520	24648986	0,000
E06	FUS	30044758	30044576	30045244	13,576
E06	CTRL	30044576	30045255	30045706	9,302
E06	NAV	18926115	18926125	18926126	0,000
E06	FS	17457784	17457797	17459135	0,000
E07	FUS	30040629	30040575	30041243	13,568
E07	CTRL	30040575	30041254	30041705	9,300
E07	NAV	20588694	20588707	20588709	0,000
E07	FS	19476205	19476735	19478110	0,000
E08	FUS	30042136	30042032	30044059	41,128
E08	CTRL	30042032	30044090	30045443	27,900
E08	NAV	22584531	22585508	22585512	0,000
E08	FS	21469566	21470514	21486437	0,000
E09	FUS	30059958	30059023	30061693	41,220
E09	CTRL	30059023	30061727	30063093	27,912
E09	NAV	19820359	19822566	19822574	0,000
E09	FS	18438268	18438292	18446310	0,000
E10	FUS	35056055	35055666	35056334	13,563
E10	CTRL	35055666	35056345	35056796	9,288
E10	NAV	21907331	21907341	21907346	0,000
E10	FS	20707517	20707542	20708884	0,000

Fonte: campos finais dos logs. Timestamps absolutos na referência do ESP Timer, sem correção retrospectiva. Na FUS, a referência esperada pode ser posterior ao início real. CPU última janela difere da média do ensaio.

B Rastreabilidade e material suplementar

O projeto que acompanha este relatório contém os logs em `dados/logs/`, os CSV derivados em `dados/`, os gráficos vetoriais e rasterizados em `imagens/` e as fontes bibliográficas em `references.bib`. A Tabela 13 permite localizar cada ensaio. O inventário armazena o SHA-256 integral de cada arquivo.

Tabela 13: Correspondência entre IDs e arquivos de coleta.

ID	Arquivo em dados/logs/
01	01_CUSTOM_PREEMPTIVO_80MHz_UNICORE.txt
02	02_CUSTOM_PREEMPTIVO_240MHz_UNICORE.txt
03	03_CUSTOM_COOPERATIVO_80MHz_UNICORE.txt
04	04_CUSTOM_COOPERATIVO_240MHz_UNICORE.txt
05	05_DM_PREEMPTIVO_80MHz_UNICORE.txt
06	06_DM_PREEMPTIVO_240MHz_UNICORE.txt
07	07_DM_COOPERATIVO_80MHz_UNICORE.txt
08	08_DM_COOPERATIVO_240MHz_UNICORE.txt
E01	EXTRA_01_CUSTOM_PREEMPTIVO_160MHz_UNICORE.txt
E02	EXTRA_02_CUSTOM_COOPERATIVO_160MHz_UNICORE.txt
E03	EXTRA_03_DM_PREEMPTIVO_160MHz_UNICORE.txt
E04	EXTRA_04_DM_COOPERATIVO_160MHz_UNICORE.txt
E05	EXTRA_05_DM_PREEMPTIVO_80MHz_UNICORE_SLICING_OFF.txt
E06	EXTRA_06_DM_PREEMPTIVO_240MHz_UNICORE_SLICING_OFF.txt
E07	EXTRA_07_RM_PREEMPTIVO_240MHz_UNICORE_SLICING_ON.txt
E08	EXTRA_08_RM_PREEMPTIVO_80MHz_UNICORE_SLICING_ON.txt
E09	EXTRA_09_RM_COOPERATIVO_80MHz_UNICORE.txt
E10	EXTRA_10_RM_COOPERATIVO_240MHz_UNICORE.txt

Fonte: inventário das coletas; hashes integrais em `dados/inventario.csv`.

A cópia de `codigo/main.c` representa o código consultado durante a análise; `codigo/sdkconfig_consultado.txt` representa a configuração disponível nessa consulta. Ambos auxiliam a auditoria, mas não identificam retroativamente cada binário das coletas. O SDK local consultado corresponde à revisão `fff9895c82d744c7237be8847347bdd1b07c6643`. Os trechos do driver/macro usados para interpretar o yield na ISR foram preservados em `fontes/trechos_sdk_consultado.txt`.

O enunciado completo está em `fontes/enunciado.pdf`, e a cópia local do exemplo inicial do professor em `fontes/Drone_exemplo_oTask_referencia.txt`. O endereço do repositório original é citado nas referências [4]. Os documentos online do ESP-IDF identificados como *latest* podem evoluir; o exemplo touch foi referenciado na revisão do SDK consultado [5].

As tabelas podem ser regeneradas a partir dos CSV incluídos executando `pythonferramentas/gerar_tabelas.py`. Essa etapa é externa ao firmware e não interfere nas medições. O pacote contém somente as observações existentes e suas derivações; não inclui ensaios sintéticos para preencher combinações não executadas.